

Deep Dive — Module 3: React (Frontend Framework)

***Goal:** understand how the user-facing layer is built — components, JSX, props, state, effects, API calls with axios, routing between pages, and the component tree — all tied to the project's Upload / Search / Dashboard / Alerts screens.*

Where React sits: It's the **only piece the user touches**. It renders the upload form, processing stepper, search results, dashboard, and alerts; it sends HTTP requests (with the JWT) to the Spring Boot API and re-renders when responses arrive. React never talks to Qdrant or PostgreSQL directly — always through the backend.

1. The core idea: declarative UI

You don't manually update the DOM. You describe what the UI *should look like for the current data* (state), and React updates the screen when the data changes. Change state → React re-renders. That's the whole mental model.

2. Components

Everything is a **component**: a function returning JSX. Components are composable and reusable.

```
function DocumentCard({ filename, score }) {
  return (
    <div className="card">
      <p>{filename}</p>
      <p>Similarity: {(score * 100).toFixed(1)}%</p>
    </div>
  );
}
```

The project's components include `UploadDocument`, `SearchResults`, `Dashboard`, `AlertsScreen`, `DocumentDetail`, and small reusable ones like `ScoreRing` and

3. JSX

JSX *looks* like HTML but is JavaScript. Key rules:

- Put JS expressions in `{ }`: `{status.toUpperCase()}`, `{score > 0.9 ? 'Dup' : 'OK'}`.
- Use `className` (not `class`) and `htmlFor` (not `for`).
- Components must return a single root element (or a `<>` fragment).
- Conditional render with `&&` or ternary: `{loading && <p>Processing...</p>}`.
- Render lists with `.map(. )` and a stable `key`.

```
{results.map(r=> <p key={r.documentId}>{r.filename}</p>)}
```

The `key` must be stable and unique (use the document id, not the array index where possible) so React can track items efficiently across re-renders.

4. Props (parent → child data)

Props are inputs passed into a component, like function arguments. Data flows *one direction*: parent down to child. To send data *up*, the parent passes a callback function as a prop.

```
function SearchPage() {
  const [results, setResults] = useState([]);
  return (
    <>
      <SearchForm onResults={setResults} />    {/* child calls onResults(.) to send data up */}
      <SearchResults results={results} />      {/* child receives data down */}
    </>
  );
}
```

5. State (`useState`)

State is data owned by a component that, when changed via its setter, triggers a re-render.

```
import { useState } from 'react';

function UploadDocument() {
  const [file, setFile] = useState(null);
  const [loading, setLoading] = useState(false);
  const [results, setResults] = useState([]);
  //.. setFile(selected), setLoading(true), setResults(data)..
}
```

Rules: never mutate state directly (`results.push(x)` ✗); always call the setter with a *new* value (`setResults([...results, x])` ✓). State updates are asynchronous and batched.

6. Calling the API (`axios` + `async/await`)

The frontend uses **axios** to call the backend, attaching the JWT in the `Authorization` header. File uploads use `FormData` (multipart).

```
import axios from 'axios';

const search = async () => {
  setLoading(true);
  const formData = new FormData();
  formData.append('file', file);
  const res = await axios.post('/api/documents/search', formData, {
    headers: { Authorization: `Bearer ${token}` }
  });
  setResults(res.data.results); // res.data is the backend JSON
  setLoading(false);
};
```

Always wrap in `try/catch/finally` so a failed request still clears the loading state and can show an error. The `/api` prefix is typically proxied to `http://localhost:8080` in development.

7. Effects (`useEffect`)

`useEffect` runs side effects (like data fetching) after render. The **dependency array** controls when it re-runs.

```
useEffect(() => {
  axios.get('/api/documents').then(res => setDocuments(res.data.content));
}, []); // [] = run once on mount
```

- `[]` → once when the component mounts (e.g. load the document list).
- `[query]` → re-run whenever `query` changes (e.g. re-fetch when a filter changes).
- Return a cleanup function to cancel subscriptions/timers when the component unmounts.

***Pitfall:** omitting the dependency array makes the effect run after every render — easy way to create an infinite fetch loop.*

8. Routing (`react-router-dom`)

React Router swaps screens **without a full page reload** (single-page app).

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

<BrowserRouter>
  <nav>
    <Link to="/upload">Upload</Link><Link to="/search">Search</Link>
    <Link to="/alerts">Alerts</Link>
  </nav>
  <Routes>
    <Route path="/" element={<Dashboard />} />
    <Route path="/upload" element={<UploadDocument />} />
    <Route path="/search" element={<SimilaritySearch />} />
    <Route path="/alerts" element={<AlertsScreen />} />
    <Route path="/docs/:id" element={<DocumentDetail />} /> {/* :id is a URL param */}
  </Routes>
</BrowserRouter>
```

Read the `:id` param inside `DocumentDetail` with `useParams()`. RBAC still matters here: hiding an `/alerts` link is *not* security — the backend must reject unauthorized roles

regardless of what the UI shows.

9. The component tree & data flow

A React app is a **tree**; state lives as high as it needs to and flows down as props.

```
App
├── Navbar
├── Dashboard
│   ├── StatCard (docs today) / StatCard (duplicates)
│   ├── RecentUploadsTable
│   └── AlertsPanel → AlertCard ×N
├── UploadDocument
│   ├── DropZone (react-dropzone)
│   └── ProcessingStepper → Step (Extract / Chunk / Embed / Store)
└── SearchResults → ResultCard ×5 → ScoreRing + MetadataPanel
```

When the search returns, `SearchResults` re-renders with new props; React efficiently updates only the changed `ResultCard` s.

10. Other pieces the project uses

- **react-dropzone** — drag-and-drop file upload area (the `DropZone`).
- **Tailwind CSS** (optional) — utility classes for styling.
- **Accessibility (WCAG 2.1 AA target):** ARIA labels on upload inputs, result lists, and alert panels; full keyboard navigation; 4.5:1 contrast. These are real project requirements, not nice-to-haves.

11. Common pitfalls

- **Mutating state directly** instead of using the setter with a new object/array.
- **Missing/incorrect `useEffect` dependency array** → stale data or infinite loops.
- **Array index as `key`** in dynamic lists → wrong items re-render; use stable ids.

- **Storing JWT in `localStorage`** → XSS risk; the project keeps it in memory/ `sessionStorage` .
 - **Treating hidden UI as access control** → always enforce roles on the backend too.
 - **Forgetting `await` /error handling** on axios calls → unhandled promise rejections and stuck spinners.
 - **Putting business logic in components** → keep it thin; the backend owns the rules.
-

12. Practice exercises

Scaffold with `npm create vite@latest` (React) or Create React App.

1. Build a `DocumentCard` component that takes `filename` and `score` props and shows the score as a percentage.
 2. Build `UploadDocument` with `file` / `loading` state, a file input, and a button that sets loading true.
 3. Add an axios `search()` that posts `FormData` to `/api/documents/search` with a Bearer header, then stores `res.data.results` in state. Add `try/catch/finally` .
 4. Build `DocumentList` that fetches `/api/documents` in `useEffect([])` and renders the list.
 5. Lift state: make a `SearchPage` parent holding `results` , passing `setResults` to a `SearchForm` child and `results` to a `SearchResults` child.
 6. Add routing for `/upload` , `/search` , `/alerts` , and `/docs/:id` ; read the id with `useParams` .
 7. Render a results table with score, vendor, and a colored label derived from the score (STRONG/RELATED/WEAK).
 8. Add a loading spinner and an error message that appear conditionally based on state.
-

13. Self-check questions

- What triggers a re-render in React?
- How does data flow between parent and child components, and how do you send data upward?

- Why must you use the state setter instead of mutating state?
 - What does the `useEffect` dependency array control, and what breaks if it's wrong?
 - Why is hiding a nav link insufficient for access control?
 - Why is a stable `key` important in mapped lists?
 - Where is the JWT stored and why not `localStorage` ?
-

14. Glossary

- **Component** — reusable function returning JSX.
 - **JSX** — HTML-like syntax compiled to JavaScript.
 - **Props** — read-only inputs passed parent → child.
 - **State** — component-owned data; changing it re-renders.
 - **Hook** — `useState` , `useEffect` , etc.; functions that add features to components.
 - **SPA** — single-page application; routing without full reloads.
 - **FormData** — browser object for multipart file uploads.
-

Navigation: [← Module 2 Qdrant](#) | [Index](#) | [Module 4 Spring Boot →](#)